BUT3 GEA — IUT de Saint-Denis

BDD SQL

Séance 3 — compter, additionner, regrouper

Séance en ligne — lundi 19 octobre

Au programme aujourd'hui

01 COUNT, SUM, AVG, MIN, MAX

02 GROUP BY : un résultat par groupe

03 GROUP BY après une jointure

04 HAVING : filtrer les groupes

05 Devoir 3, sur votre base individuelle

La séance 2, en un coup d'œil

DISTINCT, ORDER BY

Supprimer les répétitions, trier les résultats.

Calculs, AS, ROUND

Une colonne calculée, avec un nom, arrondie.

Clés et JOIN

Relier deux tables par un identifiant commun.

On va s'en servir aujourd'hui : AS, ORDER BY, ROUND et JOIN reviennent partout.

01 — Fonctions d'agrégation

Une colonne → une valeur

```
SELECT COUNT(*), SUM(Total), ROUND(AVG(Total), 2)
FROM invoices;
→ 412 | 2328.6 | 5.65
```

Fonction	Résultat
COUNT(*)	nombre de lignes
SUM(colonne)	somme
AVG(colonne)	moyenne
MIN(colonne), MAX(colonne)	plus petite, plus grande valeur

02 — Regrouper

GROUP BY : une ligne par groupe

```sql
SELECT BillingCountry,
       SUM(Total) AS ca_pays
FROM invoices
GROUP BY BillingCountry
ORDER BY ca_pays DESC;
```

Comme un TCD

GROUP BY = le pays en **lignes**.

SUM(Total) = Total en **valeurs**, avec « Somme ».

Sans GROUP BY, il faudrait une requête WHERE par pays — 24 requêtes.

La règle à retenir

Dans le SELECT : seulement les colonnes du GROUP BY, et des fonctions d'agrégation.

✗ `SELECT BillingCountry, BillingCity, SUM(Total) … GROUP BY BillingCountry`

Quelle ville afficher pour les USA ? Il y en a 13. SQLite en prend une au hasard, sans prévenir.

03 — Après une jointure

Combien chaque client a-t-il dépensé ?

```sql
SELECT customers.CustomerId, customers.LastName,
       SUM(invoices.Total) AS total_depense
FROM invoices
JOIN customers ON invoices.CustomerId = customers.CustomerId
GROUP BY customers.CustomerId, customers.LastName
ORDER BY total_depense DESC;
```

Pourquoi regrouper par l'identifiant ?

Deux clients différents peuvent porter le même nom. L'id (clé primaire) est unique. Dans vos bases individuelles, il y a des homonymes !

04 — Filtrer les groupes

WHERE ou HAVING ?

WHERE

filtre les **lignes**, **avant** de faire les groupes.

HAVING

filtre les **groupes**, **après** les avoir faits.

```sql
SELECT BillingCountry, COUNT(*) AS nb
FROM invoices
WHERE Total > 10                    -- les factures de plus de 10 $
GROUP BY BillingCountry
HAVING COUNT(*) >= 5;              -- les pays qui en ont au moins 5
```

L'ordre des clauses

`SELECT`	quelles colonnes, quels calculs ?
`FROM`	quelle table ?
`JOIN … ON`	quelle autre table, reliée comment ?
`WHERE`	quelles lignes garder ?
`GROUP BY`	quels groupes faire ?
`HAVING`	quels groupes garder ?
`ORDER BY`	dans quel ordre afficher ?

Toujours dans cet ordre. Seuls SELECT et FROM sont obligatoires.

05 — À rendre

Devoir 3

Les bases

Partie A : chinook.db.
Partie B : votre base individuelle (celle du Devoir 2).

Le rendu

Comme au Devoir 2 : une ligne **-- exercice N**, puis la requête ; un nombre ou une phrase en commentaire.

Attention

Vérifiez dans l'éditeur SQL sur quelle base la requête s'exécute.

`Prénom-NOM-SQL-Devoir3.txt → Teams`